# CSCI2010U MIDTERM - SIDE A: HOW TO ANSWER, TRACE RECIPES, JAVA BASICS, CODE TEMPLATES, BIG-O

## A1. HOW TO ANSWER EVERY QUESTION TYPE (read this first)

**"What does it print / return?"** Do NOT guess. Draw a table: one row per loop pass or per call, one column per variable. Run the code line by line. Print order = order the println lines run.

**"Fill in the blank (code)"** Look at the lines around the blank, then copy the matching template from this sheet. Check 4 things: start value, stop test, step (i++), variable names.

**"Complete the method"** 1) say what it must return, 2) variable to hold the answer (count / sum / best), 3) loop or recursion, 4) return AFTER the loop. Test your code on the example in the question by hand.

**"Big-O / running time"** Use the rules in A5. Then drop constants and small terms.

**"Explain / True-False / why"** Use the one-line answers in B8.

## A2. TRACE RECIPES

### Loop trace (arrays)

Columns: `i | a[i] | test true? | variable after`. Start i=0. Last pass is i = length-1. Loop ends when test is FALSE (i = length). Use `i < a.length` (NOT <=, that crashes: ArrayIndexOutOfBoundsException).

```
count elements > 4 in {5,2,9,2}
i=0 5>4 yes count=1
i=1 2>4 no  count=1
i=2 9>4 yes count=2
i=3 2>4 no  count=2   -> return 2
```

### Recursion trace (the #1 exam topic)

Write the first call with its numbers. Each deeper call goes one indent right. Mark the BASE case when it returns.

Then go back UP: each call finishes with the value that came back. A paused call resumes on the line AFTER its recursive call.

**Print BEFORE the recursive call = prints going DOWN (3,2,1). Print AFTER = prints coming UP (1,2,3).**

Number of calls for a countdown from n to 0 = n+1.

```
sumTo(3)              sumTo(n): if n==0 return 0
 3 + sumTo(2)                   return n +
sumTo(n-1)
     2 + sumTo(1)
         1 + sumTo(0)
             sumTo(0) = 0   <- BASE
         = 1+0 = 1
     = 2+1 = 3
 = 3+3 = 6

mystery(3): if n==0 return; print n;
mystery(n-1); print n
prints 3 2 1 [base n=0, prints nothing] 1 2 3
```

## A3. JAVA BASICS FOR NON-CODERS

Array: boxes numbered from **0**. `int[] a={5,2,9,2};` a[0]=5, a.length=4 (no brackets), last index = a.length-1 = 3. Asking a[4] crashes.

String: `s.length()` (with brackets), `s.charAt(i)` = letter at i (starts 0), `s.substring(1)` = from index 1 to end ("hello"→"ello"), `s.substring(1,3)` = index 1 up to but NOT including 3 ("el"). `s1.compareTo(s2)`: negative if s1 comes before s2, 0 equal, positive after. Case matters ("Level" differs from "level").

`n % 10` = last digit (5072 % 10 = 2). `n / 10` = drops last digit (5072 / 10 = 507, whole-number division). `count++` = add 1. `i--` = subtract 1. `==` compares, `=` assigns. `&&` and, `||` or, `!=` not equal. `countdown(--i)` subtracts 1 FIRST then passes it.

`javac Foo.java` = compile source to Foo.class (bytecode). `java Foo` = run main in Foo.class (JVM). Compile after every edit. `public static void main(String[] args)`: public = JVM can call it, static = no object needed, void = returns nothing, args = command-line words.

`import java.util.Arrays;` then `Arrays.toString(a)` prints [4, 3, 5]. Printing an array directly gives garbage like [I@251a69d7. `Arrays.sort(a)` sorts. java.lang (System, Math, String) is auto-imported.

Passing to methods: primitives (int, double) = a COPY. Arrays/objects = a copy of the REFERENCE, so the method changes the SAME array. That is why bubble sort needs no return value.

Compile error (javac says no): missing ;, wrong type, method does not exist, `new` on an abstract class, private field used outside its class, abstract method not implemented, `Manager m = new Employee()`. Runtime error (crashes while running): ArrayIndexOutOfBoundsException, IndexOutOfBoundsException (list), StackOverflowError (recursion with no base case).

Loop forms: `for (init; test; step) {body}` runs init once, then test, body, step, test... until test is false. Enhanced: `for (Employee e : arr)` = for each element.

## A4. CODE TEMPLATES (copy the matching one)

### Counting loop (Lab 1)

```
int count = 0;                    // before loop
for (int i = 0; i < a.length; i++) {
  if (a[i] >= target) count++;  // >= : equal counts
}
return count;                     // after loop
```

### Max / sum

```
int best = a[0];                  // not 0
for (int i = 1; i < a.length; i++)
  if (a[i] > best) best = a[i];
int sum = 0;
for (int i = 0; i < a.length; i++) sum += a[i];
```

### Swap needs a temp (else value lost)

```
static void swap(int[] A, int j, int k){
  int temp = A[j];
  A[j] = A[k];
  A[k] = temp; }
```

### Bubble sort (L1/L2)

```
for (int i = A.length-1; i > 0; i--) // i fixed
  for (int j = 0; j < i; j++)        // j moves
    if (A[j+1] < A[j]) swap(A, j, j+1);
// n-1 passes; compares: n-1, n-2, ..., 1
// after pass k the last k are locked
// descending: use A[j+1] > A[j]
// {5,2,4,1}: 2541 | 2451 | 2415 (5 locked)
//  pass2: 2415 | 2145 (4 locked)  pass3: 1245
```

### Linear search (L6)

```
for (int i = 0; i < A.length; i++)
  if (A[i] == val) return i;
return -1;        // best O(1), worst O(n)
```

### Binary search (L10) - array MUST be sorted

```
bs(a, val, low, high):
  if (low > high) return NOT_FOUND;
  int mid = (low + high)/2;
  if (val == a[mid]) return mid;
  else if (val < a[mid])
    return bs(a, val, low, mid-1);
  else
    return bs(a, val, mid+1, high);
start: bs(a, val, 0, a.length-1)  O(log2 n)
find 5 in {1,3,5,7,9,11,13}:
 l0 h6 m3=7 | l0 h2 m1=3 | l2 h2 m2=5 found
```

### Array insert / delete (L10), O(n) worst, O(1) best

```
insert(a, idx, val):
  for (int i=a.length-1; i>idx; i--)
    a[i] = a[i-1];      // shift RIGHT, from the end
  a[idx] = val;
delete(a, idx):
  for (int i=idx+1; i<a.length; i++)
    a[i-1] = a[i];      // shift LEFT
  a[a.length-1] = GAP;
worst idx=0 O(n), best idx=last O(1)
```

### Recursion templates (Lab 4 / L7)

```
sumDigits iter: int s=0;
  while (n>0) { s += n%10; n /= 10; }
  return s;
sumDigits rec: if (n==0) return 0;
  return n%10 + sumDigits(n/10);
countDigits: if (n<10) return 1;
  return 1 + countDigits(n/10);
sumTo: if (n==0) return 0;
  return n + sumTo(n-1);
power: if (exp==0) return 1;
  return base * power(base, exp-1);
palindrome iter: int i=0, j=t.length()-1;
  while (i<j) {
    if (t.charAt(i) != t.charAt(j)) return false;
    i++; j--; }
  return true;
palindrome rec: if (t.length()<=1) return true;
  if (t.charAt(0) != t.charAt(t.length()-1))
    return false;
  return isPal(t.substring(1, t.length()-1));
countdown(i): print i; if (i<=0) return;
  else countdown(i-1);     // 3 2 1 0
printUp(i): if (i<=0) return;
  printUp(i-1); print i;   // 1 2 3
```

### Towers of Hanoi (L7)

```
TOH(n, start, goal, temp):
  if (n == 0) return;             // base
  TOH(n-1, start, temp, goal);    // n-1 to temp
  print start + " --> " + goal;   // move 1 disc
  TOH(n-1, temp, goal, start);    // n-1 onto goal
call: TOH(n, "A", "C", "B")
n=2: A-B, A-C, B-C
n=3 (7 moves): A-C, A-B, C-B, A-C,
  B-A, B-C, A-C
moves = 2^n - 1; calls with n=0 (n=3): 8
```

## A5. BIG-O RECIPE

- No loop / fixed steps = **O(1)**. One loop 0..n = **O(n)**. Loop inside loop (both to n, or inner bounded by outer) = **O(n^2)**. Counter halved or doubled each step = **O(log n)** (log base 2). Two loops one after the other = ADD, keep the biggest: n + n = O(n).
- Divide-and-conquer with linear combine (merge sort style) = O(n log n). Nested n*n*n = O(n^3).
- **Drop constants and small terms:** 6n^2+30n+11 = O(n^2). 3+5n^2+7n^3 = O(n^3).
- Loop header test runs ONE MORE time than the body: `for(i=0;i<n;i++)` header n+1, body n.
- Comparing all pairs: (n-1)+(n-2)+...+1 = n(n-1)/2 = O(n^2).
- Growth, slow to fast: **1, log n, sqrt n, n, n log n, n^2, n^3, 2^n, n!** 2^n and n! are impractical except tiny n.

### Complexity table (all course algorithms so far)

| algorithm | best | worst |
|---|---|---|
| array access a[i] | O(1) | O(1) |
| linear search | O(1) | O(n) |
| binary search (sorted) | O(1) | O(log n) |
| array insert / delete | O(1) at last | O(n) at 0 |
| bubble sort (nested loops) | - | O(n^2) |
| perfect square v0 / v1 / v2 | - | O(n) / O(sqrt n) / O(log n) |
| sumTo(n) recursion | - | n+1 calls = O(n) |
| sumDigits / countDigits | - | 1 call per digit = O(log n) |
| Towers of Hanoi | - | 2^n - 1 moves (exponential) |
| ArrList get | O(1) | O(1) |
| ArrList add / addLast | - | O(n) (n = capacity) |

### Misc facts

- Timing code: `long s=System.nanoTime(); sort(data); long e=System.nanoTime(); (e-s)/1e9` = seconds. Timing depends on the computer and the input, so we use Big-O instead.
- Create arrays: `int[] b = new int[5];` (all start at 0) or `int[] c = {10,20,30};`. One type only, length fixed once created, index 0 to length-1. Array of objects: `Person[] staff = {boss, worker};`
- Flowchart shapes: rectangle = a step, diamond = yes/no question, arrows = flow. UML class box: name, fields (- private), methods (+ public).

## A6. RECURSION FACTS (L7, Lab 4)

- Recursion = defining something in terms of itself; a method that calls itself. Needs **BASE CASE** (a conditional, first statement, has a `return`, ends the chain) and **RECURSIVE CASE** (calls itself on a SMALLER problem so it moves to the base). Missing base case or no progress = StackOverflowError.
- Calls are stored on the **call stack**; each call keeps its own variables; a return removes it and gives the value to its caller. Overhead exists. Iterative is the default; use recursion if the problem is naturally recursive (TOH recursive, factorial iterative). Every recursive solution can be iterative.
- Solve with recursion: write base case first, think of smaller same-shaped subproblems, trust the call, combine the result. A loop stopping test = the recursive base case. String is empty OR a char + another string; a list is a basic object OR an item + the rest.
- Boxes example: v1 iterative with a pile (while loop), v2 recursive (method calls itself on each inner box).

# CSCI2010U MIDTERM - SIDE B: OOP, INHERITANCE, ABSTRACT/INTERFACE, ANALYSIS, RECURSION, LISTS, EXPLAIN-WHY

## B1. OOP VOCABULARY AND FACTS (L3)

**Class** = blueprint/template. **Object / instance** = a real thing made from it (`new`). Cookie cutter vs cookies. Instantiate = make an instance.

Every object has **STATE** (fields: what it HAS), **BEHAVIOUR** (methods: what it DOES), **IDENTITY** (what makes it unique even with same state, set by Java; `Object.equals`). **Members** = fields + methods.

Find classes: **nouns = classes, verbs = methods** (oven, cutter, baker; bake, cut, taste). Good class candidate: something with many instances that each have state + actions (wheel, window, engine, radio).

**Encapsulation** = hide data. Fields `private`, access through public methods (getters read: `getName()`, setters write: `setBonus()`). State change only through a method. `final` = cannot change. `this` = the current object (`this.name = name`). Java catches errors at COMPILE time (Python mostly at runtime).

Constructor: same name as the class, no return type, runs on `new`. All non-static classes need one.

**Inheritance** = subclass gets superclass fields/methods. Keyword **extends**. Test: **"is a"**. Manager IS AN Employee so `Manager extends Employee`. NEVER the reverse. Inheritance flows general to specific only: Manager can do all Employee does plus more; Employee cannot call `setBonus`.

**is-a vs has-a:** Manager IS AN Employee (use extends). A Car HAS AN Engine (a field, NOT extends). Only use inheritance when the "is a" sentence makes sense.

"Superclass" does NOT mean better. Subclass can **extend** (add) and **override** (redefine). UML: arrow from subclass up to superclass; italics = abstract. Hierarchy example: Animal > Mammal > Marsupial > Koala. Animal big 5: eat, excrete, breathe, move, reproduce. Mammals have these + glands, fur, neocortex; Animals do not all have those (lizard).

## B2. SUPER, OVERRIDE, POLYMORPHISM (L4, Lab 2)

```
public class Manager extends Employee {
  private double bonus;
  public Manager(double salary, String name){
    super(salary, name);     // FIRST line
    this.bonus = 0; }
  @Override
  public double getSalary(){
    return super.getSalary() + bonus; } }

LimitedCounter(String name, int limit){
  super(name); this.limit = limit; }
@Override public void increment(){
  if (getCount() < limit) super.increment(); }
for (Counter c : counters) c.increment();
  // one loop, no casts
```

Subclass CANNOT read a superclass `private` field (error: "field is not visible"). Use the public getter.

`getSalary()` inside the override with no `super.` calls ITSELF forever = **StackOverflowError**.

`super.method()` = call the superclass version. `super(...)` = call the superclass constructor, must be FIRST, needed because the subclass cannot set private superclass fields. `super.salary` is illegal (super is not an object; `this` is).

**Polymorphism:** reference type can be a SUPERCLASS of the object type. `Employee e = new Manager(..)` OK. `Manager m = new Employee(..)` DOES NOT COMPILE. Read right to left: "Manager is an Employee" OK, "Employee is a Manager" not OK.

One array `Employee[]` can hold Employees and Managers. A method with an Employee parameter accepts any subclass. The version that RUNS is the one belonging to the actual object (e.g. Manager.getSalary), called **dynamic method dispatch** (name only, not covered).

A superclass reference cannot call subclass-only methods: `workerBee.setBonus(5)` fails to compile.

Making an object = 3 steps: (1) declare REFERENCE variable (a signpost holding an address, type never changes), (2) create the object with `new` on the **HEAP** (Java garbage-collects unused ones), (3) link reference to object.

Lab 2 answers: after limit-2 counter increments 3 times = 2; reset = 0; one more = 1. Open (no limit) and Pair differ because the actual object decides which `increment` runs.

## B3. PACKAGES, ACCESS, ABSTRACT, INTERFACE (L5, Lab 3)

| access | public | protected | none | private |
|---|---|---|---|---|
| same class | Y | Y | Y | Y |
| same pkg subclass | Y | Y | Y | N |
| same pkg other | Y | Y | Y | N |
| diff pkg subclass | Y | Y | N | N |
| diff pkg other | Y | N | N | N |

**Package** = group of classes, like folders. First line `package name;`. Names unique per package. No package statement = default package. Compile with folders: `javac -d bin ...`, run with full path `lecture05.iface.Main`. Library = one or more packages. java.lang (System, Math, String) auto-imported; java.io input/output; java.util (Arrays, Random); java.net networking; javax.crypto encryption. Course uses mostly **public** and **private**; no modifier is bad practice; protected rarely.

- **Abstract class:** keyword `abstract`; **cannot be instantiated** (`new Person()` = compile error even though it has a constructor); mixes concrete and abstract methods; abstract method has no body, ends with `;`; a constructor can NOT be abstract; subclass must implement ALL abstract methods or be abstract itself; non-abstract = **concrete**; CAN have static methods. Use when several sibling subclasses share structure (Person to Employee and Intern).
- **Single inheritance:** a class can `extends` only ONE class (diamond problem: two parents with the same field/method, which one?). **Interface** (keyword `interface`) = set of requirements / a ROLE, all methods abstract, a class can `implements` MANY. Concrete class must write every method.
- `public class Employee extends Person implements Comparable<Employee>`

```
@Override public int compareTo(Employee o){
  return getName().compareTo(o.getName()); }
// or Double.compare(a, b) for numbers
// negative: this first, 0: equal, positive:
after
Arrays.sort(arr);  // needs Comparable
Employee(String name, String dept){
  super(name); this.department = dept; }
@Override public String getDescription(){
  return getName() + " - " + department; }
```

- Sorting order comes from compareTo, NOT insertion order. Generics `<T>` (the "diamond") restrict the type: `ArrayList<String>` holds only Strings (full topic L13).

## B4. ALGORITHMS AND ANALYSIS (L1, L6, L8, L9)

- **Algorithm** = finite (ends), deterministic (same input same output), effective, implementation (abstract enough for any language, every step precise). Pseudocode = for humans, language-free, skips boilerplate. Recipe analogy. al-Khwarizmi c.800 CE; formalised 1930s (Turing, Church).
- Efficiency: **time** (count basic steps) and **space** (memory); time matters most. Simple sorts: bubble, insertion, selection. Efficient: merge, quick, heap, shell. L2 timing: n x10 gave time x100 (0.12s, 13.5s, 1424s) = about n^2.
- T(n) = number of basic steps for input size n. Each pseudocode line costs a constant c_i; `end` and comments not counted. Linear search: `T = c1(t+1) + c2 t + c3 + c4`. Best case (first spot, t=1) = constant O(1). Worst case (missing, t=n) = `an + b` = linear O(n).
- Best = shortest run, worst = longest run, average (not covered). Course uses **worst case**: it is an upper-bound guarantee and it happens often (searching for something absent).
- Perfect square: v0 `while i<=n` = O(n); v1 `while i*i<=n` = O(sqrt n); v2 double i then binary search = O(log2 n).
- **BIG-O definition:** f(n)=O(g(n)) if there exist positive constants c and n0 with `0 <= f(n) <= c*g(n)` for all n >= n0. g is an UPPER bound. O(g) is a SET of functions. c and n0 must be positive (not 0). Example: 3+5n^2+7n^3 <= 15n^3.
- Big-O is a notation; worst case is a type of input; Big-O can describe best, worst or any case.
- Simplifying steps: ignore each statement cost, keep a*n+b, keep only the leading term (n^2 beats 90n once n>30), drop the leading constant.
- **Class challenge for 3+5n+18n^2:** O(n) FALSE, O(n^2) TRUE, O(n^3) TRUE; 3^n=O(n^3) FALSE; n^3=O(3^n) TRUE. Also 5n+20=O(n) T; n^2=O(n) F; n=O(n^2) T; 2^n=O(n^3) F; n log n=O(n^2) T.

## B6. LISTS (L11)

- A **List** = finite, **ordered** collection of elements of ONE type. Ordered = each has a **position**, NOT sorted by value. Empty = no elements. **Head** = start, **tail** = end. Sorted list = ascending values; unsorted = no relationship.
- Interface says WHAT (requirements), implementation says HOW (unsorted list search = linear, sorted = binary). Plan for stacks, queues, lists: interface, array version, linked-list version.
- Our `List<T>` methods: `add(T)` head, `add(T,int index)` middle, `addLast(T)` tail, `get(int)`, `isEmpty()`, `size()`, `toString()`.

```
class ArrList<T> implements List<T> {
 private static final int DEFAULT_SIZE = 10;
 private int maxSize;   // capacity
 private int size;      // stored now
 private T[] listArray;
 ArrList(){ this(DEFAULT_SIZE); }
 ArrList(int capacity){ maxSize = capacity;
   this.size = 0;
   listArray = (T[]) new Object[capacity]; }
 get(i): if (i<0 || i>size || i>=maxSize)
   throw new IndexOutOfBoundsException(..);
   return listArray[i];
 add(e,i): same check; shift right from end;
   listArray[i]=e; if (size<maxSize) size++;
O(n)
 addLast(e): if (size==maxSize) shift left
   (drops the head); put e at size (or
   maxSize-1 if full); if (size<maxSize)
size++;
   O(n)
 toString: "[]" if size==0, else "[a, b, c]"
```

- add at head = like array INSERT (shift right). addLast on a full list = like array DELETE (shift left). IndexOutOfBoundsException is an **unchecked** exception (extends RuntimeException): programmer error, need not be caught or declared.

## B8. ONE-LINE ANSWERS FOR "EXPLAIN WHY"

- **Why must a duration EQUAL to the target count?** "At least" means greater than OR equal, so use `>=`.
- **Why does `increment()` inside the override call itself?** Without `super.` Java calls the current class, which is the override, forever. `super.increment()` calls the superclass version.
- **`super(name)` vs `super.increment()`:** the first is the constructor call that sets the superclass private state (first line only); the second calls the superclass version of an overridden method.
- **Why can Employee not read Person's name?** It is private so only Person can touch it; use the public `getName()`.
- **Why does `new Person("x")` not compile?** Person is abstract, so it cannot be instantiated, even though it has a constructor.
- **Why must a concrete class implement every abstract method?** Otherwise the object would have a method with no body.
- **How does Comparable help Arrays.sort?** sort calls `compareTo` on the elements to decide the order; implementing it supplies that rule. **implements vs extends:** extends inherits ONE class; implements promises to write the interface methods and a class can have many.
- **Loop stop vs recursive base case:** both are the condition where the work ends, and each step must move toward it (n/10 reaches 0, string gets shorter).
- **Why does a mismatch return false immediately (palindrome)?** One differing pair of outer characters already proves it is not a palindrome.
- **Why O(log n) for binary search?** Each comparison halves the remaining range; n can be halved about log2 n times.
- **Why drop constants/lower terms?** Only the growth rate matters for large n; the leading term dominates.
- **Why worst case?** It is a guaranteed upper bound and it often really happens.
- **Why does printUp print 1 2 3?** It prints AFTER the recursive call, so output happens on the way back up the stack.